Capítulo 2: Fundamentos de Git

Este capítulo describe el ciclo de trabajo básico con Git: obtener un repositorio, registrar cambios, revisar el historial, ignorar archivos, deshacer cambios y sincronizar con GitHub.

2.1 Obtener un repositorio

Hay dos formas de empezar:

- **git init:** convierte la carpeta actual en un repositorio nuevo, creando la carpeta oculta .git, donde Git guarda todo.
- **git clone <URL>:** copia un repositorio existente, con todo su historial.

Ejemplo usado en esta actividad: git clone https://github.com/yaninm2000/manual-git-u2.git

2.2 El ciclo de vida de un archivo

Cada archivo del repositorio está *sin rastrear* (nuevo para Git), *sin modificar*, *modificado* o *preparado*. Tres comandos controlan ese ciclo:

Comando	Función
git status	Muestra el estado actual: qué archivos están sin rastrear, modificados o preparados
git add <archivo>	Prepara el archivo para la próxima confirmación (git add . prepara todos)
git commit -m "mensaje"	Guarda los cambios preparados en el historial con un mensaje descriptivo

Un buen mensaje de commit es breve, claro y describe qué se hizo, para que el historial sea comprensible.

2.3 Historial y diferencias

- **git log** muestra la lista de confirmaciones (autor, fecha, mensaje). Con --oneline se presenta una línea por commit, ideal para una vista rápida:

git log --oneline

- **git diff** muestra las diferencias entre versiones: sin argumentos compara lo modificado y aún no preparado, y también permite comparar dos commits (git diff <commit1> <commit2>).

2.4 Ignorar archivos (.gitignore)

No todos los archivos deben versionarse: temporales, contraseñas o archivos compilados. El archivo .gitignore lista los patrones que Git debe ignorar. → Add-Content .gitignore '~$*'

Otros patrones habituales: *.log (por extensión) o carpeta/ (una carpeta completa).

2.5 Deshacer cambios

- **git restore <archivo>:** descarta los cambios hechos en un archivo y lo devuelve al último estado confirmado. ⚠️ Esa modificación no se puede recuperar.
- **git restore --staged <archivo>:** saca un archivo del área de preparación sin perder sus cambios (es la sugerencia que muestra git status).

2.6 Repositorios remotos

Un remoto es una versión del repositorio alojada en Internet (aquí, GitHub). Al clonar, Git lo registra automáticamente con el nombre origin.

Comando	Función
git remote -v	Lista los remotos configurados y sus URL
git push origin main	Envía los commits locales de la rama main al remoto
git pull origin main	Descarga y fusiona los cambios del remoto en la rama actual

Conclusión

Con status, add, commit y push se cubre el flujo diario de trabajo. Complementados por log, diff y restore, ofrecen control y seguridad sobre el proyecto, y GitHub aporta la copia remota que permite compartirlo.

Referencia: Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/es/v2